

Agent Mesh SRE — User Guide v3.1

Autonomous Monitor · Parallel Approval Gates · Real-time Detection

Version 3.1 | June 2026 | API Days — Agent Mesh on Streaming World

1. Introduction

Agent Mesh SRE v3.1 demonstrates fully autonomous AI-driven Kafka SRE operations. The Monitor agent watches the cluster continuously, detects failure conditions, fires all 11 failover scenarios without any button clicks, and manages human-in-the-loop approval gates in parallel — one popup per pending scenario.

New in v3.1 vs v2.1:

- Fully autonomous Monitor polling loop — all 11 scenarios fire without human triggers
- Parallel approval gates — multiple gated scenarios show separate popups simultaneously
- Immediate history entries — scenarios appear in history as ⌚ Awaiting Approval the moment the gate opens
- Date/time stamps on all scenario history entries
- Scenario End Modal suppressed — review outcomes via the Scenario History bar
- Topic auto-healing — affected topics heal autonomously 90s after each scenario

2. What Is Agent Mesh SRE?

Agent	Role	Colour	v3.1 Addition
INTAKE	Ingests Kafka metric events	Emerald green	—
MONITOR	Autonomous MRAL loop + anomaly detection	Violet	Auto-fires all 11 scenarios; parallel approval gates
WRITER	Generates structured incident reports	Cyan	—
NOTIFY	Sends email + notification records	Orange	Fires for every autonomous scenario

3. What Happens Without Clicking Anything

From the moment the dashboard loads, the Monitor agent watches independently:

Time after load	What you see
0s	Dashboard loads. Cyan Monitor pill in TopBar shows cycle count.

30s	First poll cycle completes. ScenarioPanel MonitorStatusCard shows 'polling'.
35s (localhost) 8s (Vercel)	FIRST AUTONOMOUS SCENARIO fires. Canvas animates — no click needed.
If approval-gated	Policy Gate modal appears. Review reasoning and click Approve or Reject.
Immediately on gate open	 Awaiting Approval entry appears in Scenario History bar.
Notifications tab	Notification record added for every scenario completion.
90s after scenario	Broker state heals. Topic auto-heal MRAL fires for affected topics.
Next cycle (Vercel: 15–25s / localhost: 90–150s)	Second scenario fires autonomously.

4. Dashboard Layout

Left sidebar

- Monitor Status Card — poll cycle count, running state, detected scenarios and confidence scores
- Common Scenarios (4) — always visible with detection rings (cyan detected, amber auto-fired, purple suppressed)
- More Scenarios (6) — all visible with detection rings
- Kafka Topics panel — topic health with ⚡ Monitor alert chips when anomaly detected

Centre canvas

- Agent mesh with live MRAL animation — fires autonomously without button clicks
- Sub-agent circles: REASON (violet), ACT (orange), LEARN (green)

Right sidebar tabs

- Audit Log — [POLL] entries (cyan border) for every autonomous detection
- Approvals — pending and resolved approval decisions
- Notifications — record for every scenario completion

5. Detection Rings on Scenario Buttons

Ring	Colour	Meaning
• detected	Cyan pulse	Monitor sees anomaly; within 5-min cooldown — not re-firing yet
⚡ auto-fired	Amber glow	Monitor autonomously triggered this scenario; MRAL animation

		played
⊘ suppressed	Purple ring	Monitor detected but decided not to act (scenarios 04 and 11)
No ring	—	No detection in the last 2-minute window; broker is healthy for this scenario

Rings persist for 2 minutes after detection via the recentDetections buffer in the Monitor poll loop.

6. Human-in-the-Loop Approval Gates

Four scenarios require human approval before the Monitor executes an infrastructure-mutating action. Approval gates now work in parallel — if two gated scenarios fire close together, two separate popups appear simultaneously.

6.1 Parallel Gate Behaviour

- Each gate shows a separate card with its own scenario name, reasoning, confidence, and proposed tool call
- Approving or rejecting one card has no effect on others — each resolves its own Monitor callback
- All pending gates are visible simultaneously in the centre overlay
- The Scenario History bar shows ⌚ Awaiting Approval immediately when a gate opens
- Final ✓ Resolved or ✗ Rejected status updates when the scenario completes

6.2 The Four Gated Scenarios

Scenario	Tool Call	What Happens if Approved
Consumer Lag Spike	kafka.scaleConsumers	Monitor scales consumer group replicas; lag returns to zero
Share Group Rebalance	kafka.checkpointShareGroup	Monitor checkpoints share group offsets; queue depth normalises
Schema Registry Mismatch	kafka.updateSchemaCompatibility	Schema compatibility updated; DLQ drain begins
Under-Replicated Partitions	Partition reassignment	ISR restored; under-replication cleared

Rejected actions are fully audited. The Monitor logs the rejection with the reason and records a lesson. No cluster mutations are made.

7. Scenario History Bar

- Appears at the bottom of the dashboard, persists across page refreshes (localStorage)
- Entry appears immediately when a scenario starts — ⌚ Awaiting Approval for gated scenarios, no waiting needed
- Shows date/time stamp on each entry (short format, e.g. 11/06/26, 14:32)
- Click any entry to open the full Summary Modal with reasoning, action, lesson, and live events
- Colour-coded left border by scenario type
- Clear History button wipes all entries; history capped at 30 entries

Badge	Meaning
⌚ Awaiting Approval	Scenario is running and waiting for human approval — gate is open
✓ Resolved	Scenario completed successfully (approved or auto-executed)
✗ Rejected	User rejected the approval gate — no cluster action taken

8. Scenarios Reference

8.1 Common Scenarios — always visible

Consumer Lag Spike (KIP-848) | Approval-gated

Trigger Reason
Consumer group processing rate falls below producer write rate for a sustained period
Downstream service slowdown (DB bottleneck, GC pause) stalls consumer threads
Broker rebalance mid-consumption pauses processing while partitions are reassigned
Topic partition count too low for the consumer group size, creating hotspot partitions
• Auto-fires: Monitor detects <code>payments-consumer.lag > 10,000</code> growing over 3 consecutive samples • Action: <code>kafka.scaleConsumers</code> — scales consumer group replicas +2 • Topic healed: <code>ops.kafka.metrics.v1</code> (critical lag 18,500) heals 90s after scenario

KRaft Controller Failover (KRaft) | Auto-execute

Trigger Reason
Active KRaft controller broker process crashed or became unresponsive
Network partition isolated the controller from quorum voters

JVM out-of-memory on the controller node triggered OS process kill

Rolling restart or planned maintenance incremented the controller epoch

- Auto-fires: Monitor detects controllerEpoch increased beyond last cached value
- Action: kafka.acknowledgeFailover — logs new epoch, no cluster mutation

Share Group Rebalance (KIP-932) | Approval-gated

Trigger Reason

Share group member joined or left causing in-flight record redistribution

Consumer missed heartbeat deadline triggering group coordinator timeout

Share group queue depth exceeded configured fetch limit triggering rebalance

Broker partition leadership moved, invalidating existing share group assignments

- Auto-fires: Monitor detects share-group-1.lag > 10,000 growing over 3 samples
- Action: kafka.checkpointShareGroup — checkpoints offsets to stable position
- Topic healed: ops.requests.v1 (degraded lag 8,500) heals 90s after scenario

False-Positive Suppression (KIP-848) | Suppress — no action

Why Monitor suppresses

Normal partition rebalance during rolling consumer deployment — expected churn

Lag briefly rises during consumer startup before threads reach full throughput

Group coordinator election during broker maintenance looks like an outage

Short-lived network blip causes transient lag that self-resolves within seconds

- Monitor detects: rebalancing state + lag < 1,000 — intelligent no-action decision
- Shows ∅ suppressed ring on button — demonstrates Monitor reasoning, not blindly firing

8.2 More Scenarios — 6 additional

Schema Registry Mismatch (Avro) | Approval-gated

Trigger Reason

Producer updated Avro/Protobuf schema without backward-compatible evolution

Consumer deserialization fails with SchemaParseException on new fields

Schema registry compatibility mode changed from BACKWARD to NONE

Two producer versions writing incompatible schemas to the same topic simultaneously

- Auto-fires: Monitor detects schema-mismatch signal within 90s window

- Action: kafka.updateSchemaCompatibility — updates registry compatibility mode
- Topic healed: ops.actions.audit.v1 (degraded lag 7,800) heals 90s after scenario

Broker Disk Saturation (I/O) | Auto-execute

Trigger Reason
Log retention policy misconfigured — compacted topics accumulating without cleanup
Sudden traffic spike wrote log segments faster than disk throughput could handle
Log cleaner thread fell behind, leaving old segments undeleted
Large batch producers sending oversized messages exhausted available disk within hours
• Auto-fires: Monitor detects diskUsedPercent > 80% OR topic size sum > 500 MB • Action: automatic log cleanup and retention adjustment • Topic healed: ops.lessons.v1 (degraded lag 3,200) heals 90s after scenario

Under-Replicated Partitions (ISR) | Approval-gated

Trigger Reason
Follower broker fell behind leader due to network bandwidth saturation
Broker GC pause exceeded replica.lag.time.max.ms, dropping it from ISR
Rack-aware replica placement violated after broker failure leaves rack under-covered
Disk I/O saturation on a follower prevented it keeping up with replication throughput
• Auto-fires: Monitor detects underReplicatedPartitions > 0 sustained for 2+ samples • Action: partition reassignment to restore ISR to minimum in-sync replicas • Topic healed: ops.incidents.v1 (degraded lag 5,100) heals 90s after scenario

Producer Timeout Storm (Batch) | Auto-execute

Trigger Reason
Broker leader election taking longer than request.timeout.ms, expiring producer ACKs
acks=all with an under-replicated partition means no acknowledgement until ISR recovers
Network congestion between producer host and broker caused ACK timeout cascade
Producer batch size too large for available broker memory, triggering request queuing
• Auto-fires: Monitor detects producer-timeout signal within 90s window • Action: producer timeout auto-diagnosis and resolution • Topic healed: ops.kafka.metrics.v1 (degraded lag 4,900) heals 90s after scenario

Consumer Session Timeout (GC) | Auto-execute

Trigger Reason
Consumer JVM GC stop-the-world pause exceeded session.timeout.ms
Consumer application deadlock prevented poll loop from running within max.poll.interval.ms
Network partition between consumer and group coordinator triggered session expiry
Consumer processing oversized record batch beyond the heartbeat interval
- Auto-fires: Monitor detects consumer group state = dead OR memberCount = 0 - Action: auto-rebalance triggered to recover group membership - Topic healed: ops.kafka.metrics.v1 (degraded lag 3,800) heals 90s after scenario

Log Compaction Lag (Compact) | Auto-execute

Trigger Reason
Log cleaner thread count insufficient for volume of compacted topics
High write throughput on compacted topic caused dirty ratio to spike faster than cleanup
Compaction I/O competing with replication I/O on the same disk controller
Large number of unique keys producing high tombstone volume, slowing cleanup passes
- Auto-fires: Monitor detects compaction-lag signal within 90s window - Action: log compaction auto-remediation and cleaner thread scaling - Topic healed: ops.lessons.v1 (degraded lag 2,400) heals 90s after scenario

Partition Imbalance | Suppress — no action

Why Monitor suppresses
Preferred leader election skipped after broker restart, leaving leadership uneven
Rack-aware assignment drifted after multiple broker replacements
Manual partition reassignment left leaders concentrated on fewer brokers
Auto-leader-rebalance disabled while one broker had disproportionate leadership
- Monitor detects broker count change — logs detection, takes no action - Shows ∅ suppressed ring — demonstrates intelligent restraint for benign imbalance

9. Topic Auto-Healing

90 seconds after each scenario's anomaly is injected, the Monitor triggers a dedicated MRAL healing cycle for affected topics. No button press required.

Topic	Heals After Scenario	Condition Before Heal	Why Healing Needed
-------	----------------------	-----------------------	--------------------

ops.kafka.metrics.v1	Consumer Lag Spike, Producer Timeout, Consumer Session Timeout	Critical lag 18,500 / Degraded lag 4,900 / Degraded lag 3,800	Metrics pipeline backed up during Monitor anomaly detection burst; insufficient partition count for parallel consumer scaling
ops.requests.v1	Share Group Rebalance	Degraded lag 8,500	Rebalance disrupted in-flight operator request processing; assignment changed mid-flight causing duplicate deliveries
ops.actions.audit.v1	Schema Registry Mismatch	Degraded lag 7,800	Audit schema incompatibility caused consumers to reject records; DLQ drain required after schema fix
ops.lessons.v1	Disk Saturation, Log Compaction Lag	Degraded lag 3,200 / Degraded lag 2,400	Compacted topic accumulated dirty segments faster than cleaner processed; disk pressure paused log cleaner letting tombstones accumulate
ops.incidents.v1	Under-Replicated Partitions	Degraded lag 5,100	Incident records written to under-replicated partitions with reduced durability; leadership migration caused brief consumer gap

10. Monitor Status Indicators

Indicator	Location	What it shows
Cyan pulse pill	TopBar	Poll loop running, cycle count, last detected scenario count
MonitorStatusCard	Top of left scenario panel	Cycle count, last poll timestamp, detected scenarios with confidence %
Detection rings	Each scenario button	cyan = detected, amber = auto-fired, purple = suppressed
[POLL] audit entries	Right panel Audit Log	Every detection event with scenario ID, confidence, cause — cyan left border
 topic chips	Topic rows	Monitor alert when poll detects a scenario affecting that topic
 Awaiting Approval	History bar entry	Gate is open, awaiting human decision — appears immediately on trigger

11. Environment Variables

Variable	Required	Description
KAFKA_MODE	Yes	real or mock (lowercase). Activates KafkaJS/Aiven connections.
KAFKA_BOOTSTRAP	REAL only	Aiven/Confluent bootstrap host:port
KAFKA_USERNAME / PASSWORD	REAL only	SASL credentials
KAFKA_SASL_MECHANISM	REAL only	scram-sha-256, scram-sha-512, or plain
KAFKA_CA_CERT_BASE64	Aiven only	Base64-encoded CA PEM certificate
AIVEN_TOKEN	REAL only	Aiven API token for REST API calls (disk metrics, topic sizes)
AIVEN_PROJECT / SERVICE	REAL only	Aiven project and service names
NOTIFICATION_EMAIL	Optional	Recipient for autonomous scenario email notifications
SMTP_HOST / USER / PASS	Optional	SMTP credentials (Gmail App Password for SMTP_PASS)
GOOGLE_CLIENT_ID / SECRET	Yes	Google OAuth credentials
NEXTAUTH_SECRET / URL	Yes	JWT signing secret and full deployment URL

12. Troubleshooting

Issue	Cause	Fix
No autonomous MRAL after 60s	SSE not connected	Open dashboard — SSE connects and starts both loops. Check terminal for [AnomalySim] Started.
Only Consumer Lag Spike on Vercel	nextIndex resets to 0 on each serverless reconnect	Deploy latest: random start index + shorter Vercel intervals (8s first fire, 15-25s cycles)
Approval gate not appearing	add_pending_approval had no reducer handler	Apply patch-20. Now uses _globalPendingApprovals array with direct state dispatch.
Parallel gates not showing	pendingApprovals: [approval] was replacing array	patch-20 uses module-level accumulation array — each scenario appends its approval
 entry not in history	pending emailSummary dispatch not firing	Ensure patch-20 applied. Uses approval.scenarioId to build label — no scope dependency.
Timestamps missing from history	completedAt inside dispatch wrapper not	Ensure completedAt is inside the data: {} object in sendEmail dispatch

	data object	
Scenario modal not opening on click	viewHistorySummary block was removed	Dashboard should have viewHistorySummary && ScenarioEndModal rendering restored
SMTP skipped on Vercel	SMTP env vars not set	Add SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, NOTIFICATION_EMAIL in Vercel settings
Port 3002 already in use	Old dev server running	lsof -ti :3002 xargs kill -9 && npm run dev
Webpack 404 on load	Stale .next cache	rm -rf .next && npm run dev